

SQL/PLSQL and Apex

❖ Cursor

- In Oracle PL/SQL, a **cursor** is a pointer to a context area where SQL statements are processed. Cursors allow you to retrieve multiple rows from a SELECT query one row at a time.
 - **Implicit Cursor** – Automatically created by Oracle when a SQL statement is executed (e.g., INSERT, UPDATE, DELETE, or single-row SELECT).
 - **Explicit Cursor** – Defined by the programmer to handle multi-row queries.
- Cursor Attributes:
 - %ISOPEN TRUE if the cursor is open
 - %FOUND TRUE if the last FETCH returned a row
 - %NOTFOUND TRUE if the last FETCH did not return a row
 - %ROWCOUNT Number of rows fetched so far

❖ Package

- In Oracle PL/SQL, a **package** is a group of related procedures, functions, variables, cursors, and other package constructs grouped together as a single unit. Packages help in modularizing and organizing PL/SQL code efficiently.
 - Components of a Package:
 - **Package Specification** – Declares the public items (procedures, functions, variables, cursors, etc.).
 - **Package Body** – Defines the code for procedures and functions declared in the specification.
- Benefits of Packages:
 - Better code organization
 - Encapsulation of logic
 - Improved performance (loaded into memory once)
 - Easier maintenance and reuse
- **We cannot directly define a trigger *inside* a package** in Oracle PL/SQL.
- Create a **trigger** that **calls a procedure** defined in a **package**.

❖ Trigger

- A **trigger** is a stored PL/SQL block that is **automatically executed** (fired) when a **specific DML event** (INSERT, UPDATE, DELETE) occurs on a table or view.
- **Main Purposes:**
 - Enforce Business Rules Automatically ensure business logic (e.g., prevent negative salaries).
 - Audit Data Changes Track changes (who, when, what) for INSERT, UPDATE, DELETE operations.
 - Automatically Generate Values Auto-fill columns like timestamps, created_by, etc.
 - Prevent Invalid Transactions Stop or modify DML operations if certain conditions are not met.
 - Log Events Record actions to an audit or log table.

- Maintain Related Data Keep consistency between related tables (e.g., after insert into parent, update child).
- One Trigger per Timing/Event/Level Combination:
 - Timing (BEFORE, AFTER)
 - Event (INSERT, UPDATE, DELETE)
 - Level (ROW, STATEMENT)
- Basic Syntax:


```
CREATE [OR REPLACE] TRIGGER trigger_name
{BEFORE | AFTER | INSTEAD OF}
{INSERT | UPDATE | DELETE}
ON table_name
[FOR EACH ROW]
BEGIN
  -- Trigger logic here
END
```
- Types of Triggers:
 - BEFORE Fires before the DML event
 - AFTER Fires after the DML event
 - INSTEAD OF For views (not tables)
 - ROW-level FOR EACH ROW – fires once for each row
 - Statement-level Default – fires once per SQL statement
 - A **statement-level trigger** in Oracle PL/SQL fires **once per DML statement**, regardless of how many rows are affected. This is different from a **row-level trigger**, which fires **once for each row** affected.
 - When to Use a Statement-Level Trigger:
 - ◆ Log or audit that a DML operation occurred (not the specific rows).
 - ◆ Enforce rules or validations before/after a large operation.
 - ◆ Initialize or reset control data once per DML action.
 - Syntax:


```
CREATE [OR REPLACE] TRIGGER trigger_name
{BEFORE | AFTER} {INSERT | UPDATE | DELETE}
ON table_name
BEGIN
  -- Code here runs once per DML statement
END;
```
 - **Note:** No FOR EACH ROW clause = **statement-level** trigger.
- Important Notes:
 - Use: OLD to refer to existing values.
 - Use: NEW to refer to new values (only in INSERT or UPDATE).
 - Don't commit or rollback inside a trigger.
 - Notes:
 - You can access column values like: OLD.column_name or :NEW.column_name.
 - : NEW can be **modified** in **BEFORE** triggers (not in **AFTER**).
 - : OLD is **read-only** and cannot be modified.

Keyword	Purpose	Used In
:OLD	Before values	UPDATE, DELETE
:NEW	After values	INSERT, UPDATE

➤ Difference:

Difference Between Statement-Level and Row-Level Triggers

Feature	Statement-Level Trigger	Row-Level Trigger
Execution Frequency	Once per SQL statement	Once for each affected row
Access to :OLD / :NEW	✗ Not available	✓ Available (:OLD , :NEW)
Syntax	FOR EACH ROW is not used	Uses FOR EACH ROW clause
Performance	Faster (less overhead)	Slower (more overhead due to per-row execution)
Use Case	General auditing, logging DML actions	Tracking specific row changes, enforcing logic
Fires On Bulk DML?	Once per DML statement	Multiple times (once per row in the DML)

➤ Example :

--Example Of Trigure

```
CREATE TABLE employees (
  emp_id   NUMBER PRIMARY KEY,
  emp_name VARCHAR2(100),
  job_title VARCHAR2(50),
  department VARCHAR2(50),
  hire_date DATE,
  salary   NUMBER(10, 2)
);
```

```
INSERT INTO employees VALUES (101, 'Alice Smith', 'Developer', 'IT', TO_DATE('15-MAR-2020', 'DD-MON-YYYY'), 60000)
/
```

```
INSERT INTO employees VALUES (102, 'Bob Johnson', 'Analyst', 'Finance', TO_DATE('20-JAN-2019', 'DD-MON-YYYY'), 55000)
/
```

```
INSERT INTO employees VALUES (103, 'Charlie Brown', 'Team Lead', 'IT', TO_DATE('01-JUL-2018', 'DD-MON-YYYY'), 75000)
/
```

```
INSERT INTO employees VALUES (104, 'Diana Prince', 'HR Manager', 'HR', TO_DATE('25-FEB-2021', 'DD-MON-YYYY'), 65000)
/
```

```
INSERT INTO employees VALUES (105, 'Ethan Hunt', 'Developer', 'IT',  
TO_DATE('10-OCT-2022', 'DD-MON-YYYY'), 58000)  
/
```

--Statement-Level

```
CREATE TABLE emp_audit (  
    action_date DATE,  
    action_type VARCHAR2(20)  
);
```

```
CREATE OR REPLACE TRIGGER trg_emp_insert_audit  
AFTER INSERT ON employees  
BEGIN  
    INSERT INTO emp_audit (action_date, action_type)  
    VALUES (SYSDATE, 'INSERT');  
END;  
/
```

```
INSERT INTO employees (emp_id, emp_name) VALUES (101, 'Alice');
```

-- OR even this:

```
INSERT INTO employees (emp_id, emp_name)  
SELECT 106, 'Bob' FROM dual  
UNION  
SELECT 107, 'Charlie' FROM dual;
```

-- Row-Level Triggers

```
CREATE TABLE salary_log (  
    emp_id    NUMBER,  
    old_salary NUMBER,  
    new_salary NUMBER,  
    changed_on DATE  
);
```

```
CREATE OR REPLACE TRIGGER trg_salary_update  
BEFORE UPDATE OF salary ON employees  
FOR EACH ROW  
WHEN (OLD.salary != NEW.salary)  
BEGIN  
    INSERT INTO salary_log (emp_id, old_salary, new_salary, changed_on)  
    VALUES (:OLD.emp_id, :OLD.salary, :NEW.salary, SYSDATE);  
END;  
/
```

```
UPDATE employees
SET salary = salary + 300
WHERE emp_id = 103;
```

❖ Variable

- Variable : In **Oracle PL/SQL**, a **variable** is a named storage location used to **store data temporarily** during the execution of a block of code.
- **Variable Types:**
 - **Global Variables: Package variables** behave like **global variables** within your session — they are **initialized once per session** and accessible from any procedure or function within the package.
 - **Scalar:**
 - **NUMBER** Numeric values v_salary NUMBER(8,2);
 - **VARCHAR2** Text strings v_name VARCHAR2(100);
 - **CHAR** Fixed-length text v_code CHAR(10);
 - **DATE** Date and time v_hire_date DATE;
 - **BOOLEAN** Logical TRUE/FALSE/NULL v_is_valid BOOLEAN;
 - **Composite:**
 - **Records (RECORD):** A record is like a row of a table.

```
DECLARE
TYPE emp_rec_type IS RECORD (
emp_id employees.employee_id%TYPE,
emp_name employees.first_name%TYPE
);
v_emp_rec emp_rec_type;
```
 - **Collections:**
 - ◆ **Associative Arrays (Index-by Tables):** Indexed by string or integer.

```
TYPE name_array IS TABLE OF VARCHAR2(100) INDEX BY BINARY_INTEGER;
v_names name_array;
```
 - ◆ **Nested Tables:**

```
TYPE num_table IS TABLE OF NUMBER;
v_nums num_table := num_table(1, 2, 3);
```
 - ◆ **VARRAYs (Variable-size Arrays):**

```
TYPE grade_array IS VARRAY(5) OF CHAR(1);
v_grades grade_array := grade_array('A', 'B', 'C');
```
 - **Reference:** These refer to other program objects or data
 - **REF CURSOR** Pointer to a query result set (used for dynamic SQL) TYPE emp_cur IS REF CURSOR;
 - **LOB** CLOB, BLOB, BFILE Handle large text/binary data
 - **Anchored** %TYPE, %ROWTYPE Dynamically link to table structure

❖ Collection

- **Associative Arrays (Index-by Tables):**
 - Unbounded, sparse.
 - Indexed by numbers or strings.

- Can be used in both SQL and PL/SQL (mostly PL/SQL).
- Syntax:
 - TYPE emp_array_type IS TABLE OF employees.first_name%TYPE INDEX BY PLS_INTEGER;
- Example:


```
DECLARE
  TYPE num_array IS TABLE OF NUMBER INDEX BY PLS_INTEGER;
  arr num_array;
BEGIN
  arr(1) := 100;
  arr(5) := 200;
  DBMS_OUTPUT.PUT_LINE(arr(1));
END;
```
- Nested Tables:
 - Can be stored in database columns.
 - Unbounded but dense (no gaps in between).
 - Require initialization using TABLE() or NEW.
 - Syntax: TYPE name_list IS TABLE OF VARCHAR2(50);
 - Example:


```
DECLARE
  TYPE name_list IS TABLE OF VARCHAR2(50);
  names name_list := name_list('Alice', 'Bob', 'Charlie');
BEGIN
  FOR i IN 1..names.COUNT LOOP
    DBMS_OUTPUT.PUT_LINE(names(i));
  END LOOP;
END;
```
- VARRAYs (Variable-size Arrays)
 - Bounded (fixed max size).
 - Dense and ordered.
 - Can be stored in tables or used in PL/SQL.
 - Syntax: TYPE grade_array IS VARRAY(5) OF NUMBER;
 - Example:


```
DECLARE
  TYPE grade_array IS VARRAY(3) OF NUMBER;
  grades grade_array := grade_array(85, 90, 95);
BEGIN
  FOR i IN 1..grades.COUNT LOOP
    DBMS_OUTPUT.PUT_LINE(grades(i));
  END LOOP;
END;
```
- Quick Comparison:

Feature	Associative Array	Nested Table	VARRAY
Indexed by	Integer/String	Integer	Integer
Size	Unbounded	Unbounded	Bounded (fixed)
Sparse	Yes	No	No
Stored in DB?	No	Yes	Yes

❖ Exception

- In **Oracle PL/SQL**, an **exception** is a runtime error or an unexpected event that disrupts the normal flow of execution. You can **handle exceptions** gracefully using the `EXCEPTION` block, so your code doesn't crash and you can provide custom logic (like logging or retrying).
- **Types of Exceptions:**
 -  **Predefined (System-defined) exceptions:** **Predefined Exceptions** in Oracle PL/SQL — these are **automatically defined** by Oracle to handle **common runtime errors** like divide-by-zero, no rows returned, invalid numbers, etc.
 - What Are Predefined Exceptions?
 - ◆ Oracle has **built-in exception names** for **common errors**.
 - ◆ These exceptions are **automatically raised** when that error occurs.
 - ◆ You can catch and handle them using `WHEN` in an `EXCEPTION` block.
 - ◆ **Common Predefined Exceptions:**
 1. `NO_DATA_FOUND` `ORA-01403` A `SELECT INTO` query returned no rows
 2. `TOO_MANY_ROWS` `ORA-01422` A `SELECT INTO` returned more than one row
 3. `ZERO_DIVIDE` `ORA-01476` Division by zero
 4. `INVALID_NUMBER` `ORA-01722` Invalid number conversion
 5. `CURSOR_ALREADY_OPEN` `ORA-06511` Cursor is already open
 6. `DUP_VAL_ON_INDEX` `ORA-00001` Duplicate value violates a unique constraint
 7. `VALUE_ERROR` `ORA-06502` Assignment problem like value too large
 8. `ACCESS_INTO_NULL` `ORA-06530` Object/record is null and accessed
 9. `CASE_NOT_FOUND` `ORA-06592` No match found in `CASE` expression
 10. `COLLECTION_IS_NULL` `ORA-06531` You tried to use a collection before initializing it
 - `SQLERRM` returns the **error message**
 - `SQLCODE` returns the **numeric error code**
 -  **User-defined exceptions:** **User-Defined Exceptions** in Oracle PL/SQL — these are custom exceptions that **you define and control** based on your business logic.
 - **Declare** the exception:
 - ◆ `vMy_exception EXCEPTION;`
 - **Optionally associate** it with a specific Oracle error using `PRAGMA EXCEPTION_INIT`
 - ◆ `PRAGMA EXCEPTION_INIT(vMy_exception, -20001);`
 - **Raise** the exception:
 - ◆ `RAISE vMy_exception;`
 - **Handle** it in the `EXCEPTION` section:
 - ◆ `WHEN my_exception THEN -- handle it`
 -  **Unhandled exceptions** (caught by `WHEN OTHERS`) : **Unhandled Exceptions** in Oracle PL/SQL — these are the **errors that occur during execution but are not caught** by your PL/SQL block's `EXCEPTION` section.

- What Is an Unhandled Exception?
 - ◆ Any error that occurs during execution,
 - ◆ And there is **no matching WHEN clause** in your `EXCEPTION` block,
 - ◆ Or you **forgot to write an EXCEPTION block** at all.

➤ Summary:

What	Description
✗ Unhandled	No <code>EXCEPTION</code> block or no match inside it
✓ Handled	Error is caught with <code>WHEN ... THEN</code>
💡 Best Practice	Always include <code>WHEN OTHERS THEN</code> for safety/logging

❖ Difference between Page and Region:

- **Page:** A **page** is the **top-level container** in an APEX application. It represents a **complete screen or a view** in your application and typically includes multiple regions that contribute to the content on that page.
 - **Key Characteristics of a Page:**
 - Container for Regions: Pages contain regions (e.g., reports, forms, charts) that define the content and layout of the page.
 - Can Have Multiple Regions: A page can have multiple regions, each serving different functions (e.g., a report region, a form region, and a chart region).
 - Defined at the Application Level: Pages are part of the overall application and are often the main navigation items.
 - Page Attributes: Each page has attributes like Page Name, Page ID, Page Type, and Page Number that help identify and configure it.
 - Navigation: Pages are typically accessed from the application's menu or through links between pages.
 - Page Life Cycle: A page is rendered when the user accesses it and undergoes a lifecycle where it processes events, runs validations, and executes actions.
- **Region:** A **region** is a **container for content** that is displayed within a page. Regions are used to display **data or interactive components** (e.g., reports, forms, charts, buttons, text, etc.) within a page.
 - **Key Characteristics of a Region:**
 - **Content Section:** Regions are used to organize the content within a page. A region can be a report, form, chart, or even a static HTML block.
 - **Located Within Pages:** A region is always inside a page. You can have multiple regions on a page, placed in different locations or layouts.
 - **Dynamic and Customizable:** Regions can be dynamically displayed/hidden based on conditions, or their content can be dynamically generated using PL/SQL, SQL, or JavaScript.
- **Region Types:** A region can have different types, such as:
 - Report Region (Interactive Report, Classic Report, etc.)
 - Form Region (Interactive Form, APEX Form, etc.)

- Chart Region (Bar, Pie, Line charts)
- HTML Region (Static content like text or HTML)
- Button Region (Buttons to trigger actions)
- **Region Positioning:** Regions are placed within **region positions** on the page (e.g., top, left, right, bottom) and can be arranged using the layout editor.
- **Key Differences Between Page and Region:**

Aspect	Page	Region
Definition	A top-level container that represents a screen or view in the application.	A section within a page used to display content or functionality (e.g., reports, forms, charts).
Purpose	To structure the main navigation and content flow within the application.	To display specific content (e.g., report, form) within a page.
Hierarchy	Pages are the primary level of the application's structure.	Regions are elements within a page, and multiple regions can exist on one page.
Location	Located at the application level and accessible through links or menu items.	Located within a page and define the content layout of the page.
Content	Pages define the layout and structure of the entire screen.	Regions contain specific content like reports, forms, charts, or buttons.
Life Cycle	Pages go through the process of rendering , processing , and validation as part of the application flow.	Regions are displayed based on page rendering and can be shown/hidden dynamically based on conditions.
Navigation	Pages are often part of the main menu or navigation flow in the application.	Regions exist within a page and contribute to the content or interaction.
Example	A page for employee management that could include a report, form, and chart.	A region that could be a Classic Report or a Form within the employee management page.

❖ Dynamic Action

- A **Dynamic Action** lets you perform **actions in response to events** on a page — like button clicks, value changes, page loads, etc.
- **Key Parts of a Dynamic Action:**

<u>Section</u>	<u>Purpose</u>
▪ Event	What triggers the action (e.g. Click, Change, Page Load)
▪ Selection Type	What component triggers the event (Item, Button, Region, etc.)
▪ Action(s)	What happens next (e.g. Set Value, Show/Hide, Refresh, Execute JS Code)
▪ Client-side Condition	Optional conditions to further control behavior
- **Common Events:**
 - Click
 - Change
 - Page Load

- Dialog Closed
- Custom Event

➤ Common Actions:

<u>Action</u>	<u>Description</u>
▪ Set Value	Set value of another item
▪ Show / Hide	Show or hide an item/region
▪ Enable / Disable	Control item state
▪ Refresh	Refresh a region (e.g. IR, IG, Chart)
▪ Execute JavaScript Code	Run custom JavaScript
▪ Execute PL/SQL Code	Run server-side logic via AJAX

❖ Validation

- In **Oracle APEX**, **validation** ensures that the data entered by the user meets certain criteria before it is processed or saved. Validations are crucial for ensuring **data integrity** and for providing **feedback** to users when the data is not valid.
- Types of Validations in Oracle APEX:
 - **Item-level validation:**
 - Item-level validations check the data entered into a specific item (e.g., text field, date picker, etc.).
 - Common Types:
 - ◆ Required Field: Ensures that an item is not left empty.
 - ◆ Format Mask: Ensures the data matches a certain format (e.g., email, phone number).
 - ◆ Value Comparison: Compares two item values (e.g., confirming password).
 - ◆ PL/SQL Function: Custom logic for validation using PL/SQL code.
 - **Page-level validation:**
 - Page-level validations apply to the whole page. These are typically used for **data consistency** or ensuring that **multiple fields meet certain criteria**.
 - **Server-side validation:**
 - Server-side validations are written using PL/SQL code and are executed on the database server when a form is submitted. These validations are more robust and can access the database for complex logic.
 - **Client-side validation:**
 - Client-side validations use JavaScript and are executed in the browser before sending data to the server. They help **reduce server load** and provide **faster feedback** to users.
 - Built-in Validations in APEX:
 - Oracle APEX provides several built-in validations, such as:
 - ◆ Required Item: Ensures an item is not empty.
 - ◆ Item Value Comparison: Compare values between two items.
 - ◆ Regular Expression Match: Validate item values using regular expressions (e.g., for emails, phone numbers).
 - ◆ SQL Query Validation: Validate based on a query (e.g., ensuring the value is in a list).
 - Validation Execution Sequence:
 - When are validations triggered?
 - ◆ Validations are executed before a form is submitted to the server.
 - ◆ They are executed after calculations and computations on the page.

- **Validation Order:**
 - ◆ Item-level validations are executed first.
 - ◆ Page-level validations are executed after item-level validations.
 - **Validation Actions:**
 - ◆ If the validation fails, the error message will appear, and the page submission will be stopped.
 - ◆ If the validation passes, the page will submit normally.
- **Summary of Key Points:**
- Validations ensure the data is correct before processing.
 - Item-level validations are for single fields, while page-level checks multiple fields.
 - Client-side validations speed up the process, while server-side ensures complex checks (e.g., database validation).
 - Built-in validations can save time for common checks (like required fields).

Oracle Apex Learning:

❖ Limitation of Apex

1. Performance Limitations

- 🔧 Shared CPU & Resources: In Oracle APEX (especially on Oracle Cloud Free Tier), multiple applications share the same database resources, which can cause performance bottlenecks.
- 🔧 Limited Scalability for Large Applications: APEX is best suited for small to medium-sized applications. Extremely high-traffic applications may require additional tuning and database optimization.

2. UI & Customization Constraints

- 🔧 Limited UI Flexibility: APEX provides a built-in set of themes and templates, but highly customized UI designs might require extensive use of JavaScript, CSS, and jQuery.
- 🔧 Lack of Native Mobile App Support: APEX applications are responsive and work in mobile browsers, but they are not true native mobile apps (e.g., iOS/Android apps).
- 🔧 Limited Built-in Charts & Graphics: APEX includes Oracle JET charts, but if you need more advanced visualizations, you may need to integrate third-party charting libraries.

3. Security Considerations

- 🔧 Requires Proper Configuration: Since APEX is web-based, security must be properly configured to prevent SQL injection, cross-site scripting (XSS), and session hijacking.
- 🔧 Dependence on Oracle REST Data Services (ORDS): APEX requires ORDS for web-based access. Improper ORDS configuration may lead to security vulnerabilities.

4. Integration Challenges

- ✚ Limited Support for External Databases: APEX works best with Oracle databases. Direct integration with MySQL, PostgreSQL, or SQL Server requires additional effort, such as using REST APIs or database links.
- ✚ No Native Support for Complex Web Services: While APEX supports REST and SOAP, complex API authentication (OAuth, JWT, etc.) may require custom PL/SQL development.

5. PL/SQL Dependency

- ✚ APEX Relies on PL/SQL: If you need complex business logic, you must write it in PL/SQL, which may be a limitation for developers unfamiliar with Oracle's procedural language.

6. Licensing & Deployment Restrictions

- ✚ Enterprise Features Require Paid Licenses: Some APEX features, like Autonomous Database or high-performance database options, require Oracle Database Enterprise Edition.
- ✚ Limited Hosting Options: APEX runs on Oracle DB, meaning you cannot deploy it on non-Oracle databases like MySQL or SQL Server.

7. Multi-User Development Limitations

- ✚ Difficult for Large Teams: APEX does not have advanced version control like GitHub or Bitbucket. Collaboration between multiple developers requires manual coordination.
- ✚ Limited Support for CI/CD Pipelines: While possible, setting up DevOps pipelines for APEX apps requires additional tools like SQLcl, Liquibase, or Flyway.

❖ Security

- **Authentication (Who can log in?):**
 - Use built-in APEX authentication (username/password)
 - Integrate Single Sign-On (SSO) or OAuth2
 - LDAP or Oracle APEX Accounts
 - Navigate to: Shared Components → Authentication Schemes
- **Authorization (What can users access?):**
 - Control access to:
 - Pages
 - Regions
 - Buttons
 - Processes
 - Steps:
 - Go to Shared Components → Authorization Schemes
 - Create roles like Admin, Manager, User
 - Apply them to page-level or component-level
- **Session State Protection (SSP):**
 - Set to "Restricted" or "Checksum Required"
 - Helps prevent users from injecting malicious values via the URL
 - Set it on **items, buttons, or branches**.

- **Page Access Control:**
 - Set pages as Public, Authenticated, or Hidden
 - Add role checks in Page Authorization
- **HTTPS/SSL:**
 - Make sure your APEX is served over **HTTPS**, not HTTP:
 - Configure at the web server or reverse proxy level (e.g., Apache, ORDS, Nginx)
 - Ensures all communication is encrypted
- **SQL Injection Prevention:**
 - Use bind variables in PL/SQL (:P1_ID, etc.)
 - Never directly concatenate user input into SQL
- **Cross-Site Scripting (XSS) Protection:**
 - Use APEX escape functions like HTF.ESCAPE_SC or APEX_ESCAPE.HTML
 - Set "Escape special characters" = Yes for display items and regions
- **Application-Level Settings:**
 - In **App Builder → Edit Application → Security:**
 - Enable Browser Security
 - Set Session Timeout and Idle Timeout
 - Turn on Deep Linking Protection
- **Developer Security:**
 - Limit access to APEX Workspace
 - Use two-factor authentication (2FA) for developers
 - Audit developer activity

❖ Dashboard Create

❖ Image Upload (Live or From Computer)

❖ Document Upload

❖ Knowledge of HTML, CSS, JavaScript, jQuery, and RESTful APIs.

❖ Ajax Call

❖ Custom Design with HTML CSS

Oracle APEX, here are some advanced areas you can focus on to take your skills further:

1. Performance Optimization

- Use **SQL Profiler** to analyze and optimize queries.
 - 1. Use APEX Debug Mode
Enable Debug Mode in Oracle APEX to analyze SQL execution.
Check execution times in the APEX Debug Logs.
Steps:
Open your APEX application.
Click on your user profile (top-right corner).

Select Debug → Set to Level 9 (ALL MESSAGES).

Run your process/page and review the logs in Developer Console.

- 2. Use SQL Trace and TKPROF

Oracle SQL Trace helps capture detailed query execution statistics.

TKPROF formats and analyzes the trace output.

Steps to Enable SQL Trace:

```
ALTER SESSION SET SQL_TRACE = TRUE;
```

Run your SQL, then disable tracing:

```
ALTER SESSION SET SQL_TRACE = FALSE;
```

Find the generated trace file in the user_dump_dest directory and analyze it using
TKPROF:tkprof your_trace_file.output.txt EXPLAIN=username/password

- 3. Use EXPLAIN PLAN

Analyze how Oracle executes a query and optimize indexing or joins.

Example:

```
EXPLAIN PLAN FOR
```

```
SELECT * FROM employees WHERE department_id = 10;
```

```
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

Look for Full Table Scans (TABLE ACCESS FULL) and replace them with indexes if needed.

- 4. Use V\$SQL for Expensive Queries

Find slow-running queries from Oracle's shared SQL area.

```
SELECT sql_text, executions, elapsed_time / 1000000 AS elapsed_seconds
```

```
FROM v$sql
```

```
WHERE elapsed_time > 10000000
```

```
ORDER BY elapsed_time DESC FETCH FIRST 10 ROWS ONLY;
```

Optimize queries with high elapsed time.

- 5. Use AWR (Automatic Workload Repository)

If AWR is enabled, generate a report for slow queries.

```
SELECT * FROM dba_hist_sqlstat ORDER BY elapsed_time_total DESC FETCH FIRST  
10 ROWS ONLY;
```

Generate an AWR report:

```
@$ORACLE_HOME/rdbms/admin/awrrpt.sql
```

Choose a snapshot range and analyze SQL performance.

- 6. Use SQL Monitoring (Active Session History - ASH)

If you have Oracle Enterprise Edition, use SQL Monitoring:

```
SELECT sql_id, sql_text, elapsed_time, cpu_time, buffer_gets
```

```
FROM v$sql_monitor
```

ORDER BY elapsed_time DESC;

Optimization Techniques

Indexing:

Create indexes on frequently queried columns.

CREATE INDEX emp_dept_idx ON employees(department_id);

Bind Variables:

Avoid hard-coded values in SQL.

SELECT * FROM employees WHERE department_id = :dept_id;

Partitioning:

Split large tables for faster access.

Avoid SELECT *:

Fetch only required columns.

Optimize Joins:

Prefer INNER JOIN over LEFT JOIN when possible.

Ensure indexed columns are used in joins.

- Optimize **PL/SQL processes** and avoid unnecessary loops.
- Implement **bind variables** in queries for better performance.
- Use **APEX Advisor** to detect performance bottlenecks.

2. Dynamic Actions & JavaScript Integration

- Master **complex Dynamic Actions** to enhance user interactions.
- Integrate **jQuery & JavaScript** for better UI/UX.
 - Ggsdg

Integrating jQuery & JavaScript in Oracle APEX for Better UI/UX

Oracle APEX supports jQuery and JavaScript for enhanced interactivity and user experience. Here's how you can integrate them effectively:

1. Add Custom JavaScript in APEX

You can add JavaScript in three places:

Global Level:

Go to Shared Components → User Interface Attributes → JavaScript File URLs

Add a reference to external JavaScript libraries if needed.

Page-Level JavaScript:

Open a page → Go to Page Attributes → Execute when Page Loads.

Write your JavaScript here.

Region-Level JavaScript:

Add JavaScript inside Dynamic Actions or directly in an HTML Region using `<script>` tags.

2. Using jQuery for UI Enhancements

jQuery is built into APEX, so you can directly use it.

Example 1: Hide/Show Regions Dynamically

js

Copy

Edit

```
$("#P1_REGION").hide(); // Hide region

$("#P1_BUTTON").click(function() {

    $("#P1_REGION").toggle(); // Toggle region visibility

});
```

#P1_REGION = ID of the APEX region

#P1_BUTTON = ID of a button

Example 2: Change Button Color on Hover

js

Copy

Edit

```
$("#P1_BUTTON").hover(  
    function() { $(this).css("background-color", "green"); },  
    function() { $(this).css("background-color", ""); }  
);
```

3. AJAX Callbacks for Dynamic Data Updates

You can use AJAX to fetch data without refreshing the page.

Step 1: Create an APEX AJAX Process

Go to Processing → Create Process

Process Type: PL/SQL

Example PL/SQL Code:

sql

Copy

Edit

DECLARE

```
v_name VARCHAR2(100);
```

BEGIN

```
SELECT emp_name INTO v_name FROM employees WHERE emp_id =  
:P1_EMP_ID;
```

```
APEX_JSON.OPEN_OBJECT;
```

```
APEX_JSON.WRITE('name', v_name);
```

```
APEX_JSON.CLOSE_OBJECT;
```

```
END;
```

Set "Process Point" to "Ajax Callback"

Give it an identifier like "GET_EMP_NAME"

Step 2: Call the Process Using jQuery AJAX

js

Copy

Edit

```
$.ajax({
    url: apex.server.process("GET_EMP_NAME"),
    type: "POST",
    data: { p_request: $("#P1_EMP_ID").val() },
    success: function(data) {
        var jsonData = JSON.parse(data);
        $("#P1_EMP_NAME").val(jsonData.name);
    }
});
```

4. Enhance APEX Forms with JavaScript

Example: Auto-Fill Data on Field Change

If you want to auto-fill a field when another field is selected:

js

Copy

Edit

```
$("#P1_DEPARTMENT").change(function() {  
    var deptName = $(this).val();  
    $("#P1_DEPT_DISPLAY").text("You selected: " + deptName);  
});
```

5. Use jQuery UI for Dialogs & Popups

Example: Show Dialog on Button Click

js

Copy

Edit

```
$("#P1_BUTTON").click(function() {  
    $("#myDialog").dialog();  
});
```

Create a Static Region with ID myDialog, and APEX will render it as a popup.

6. Enhance Interactive Grid with JavaScript

Example: Change Row Color in an Interactive Grid

js

Copy

Edit

```
$("#P1_IG").on("apexafterrefresh", function() {  
    $("td[headers='STATUS']").each(function() {
```

```
if ($(this).text() === "Completed") {  
    $(this).css("background-color", "lightgreen");  
}  
});  
});
```

P1_IG = Interactive Grid Static ID

Checks each row's "STATUS" column and changes the background color.

7. Using JavaScript in APEX Dynamic Actions

Go to Page Designer → Dynamic Actions

Create a "When Button Clicked" event.

Under "Action," select "Execute JavaScript" and enter:

js

Copy

Edit

```
alert("Button Clicked!");
```

8. Use External JavaScript Libraries

You can include libraries like Chart.js, SweetAlert, or Moment.js.

Example: Add a SweetAlert Popup

Go to Shared Components → User Interface Attributes → JavaScript File URLs

Add URL:

text

Copy

Edit

<https://cdn.jsdelivr.net/npm/sweetalert2@11>

Use in JavaScript:

js

Copy

Edit

```
Swal.fire({
  title: "Success!",
  text: "Your data has been saved.",
  icon: "success",
  confirmButtonText: "OK"
});
```

Final Thoughts

Use jQuery for simple UI tweaks.

Use AJAX for faster database interactions.

Enhance Interactive Grids and Reports with JavaScript.

Use external JavaScript libraries for better UX.

- Use **AJAX callbacks** to fetch/update data asynchronously.

3. Advanced Security Practices

- Implement **session state protection** to prevent URL tampering.
- Use **row-level security (RLS)** with Virtual Private Database (VPD).
- Configure **authentication schemes** like LDAP, OAuth2, and SSO.
- Apply **Fine-Grained Access Control (FGAC)** for data security.

4. RESTful Web Services (ORDS)

- Develop **RESTful APIs** for integrating APEX with external systems.
- Use **OAuth 2.0 & JWT authentication** in ORDS for secure API access.
- Implement **APEX Web Source Modules** for consuming external APIs.

5. Custom Plugin Development

- Create **custom region plugins** for advanced UI components.
- Develop **dynamic action plugins** for enhanced interactivity.
- Use **PL/SQL, JavaScript, and CSS** to build reusable components.

6. Automating Processes with APEX Automations

- Set up **scheduled jobs** for background processing.
- Automate **email notifications and reports** based on data changes.
- Integrate with **Oracle Workflow or BPEL** for complex business logic.

7. Advanced Jasper Reports & PDF Generation

- Use **JasperSoft Studio** to design complex reports.
- Implement **sub-reports, charts, and custom data sources**.
- Optimize **report queries** for better performance.

8. Oracle Forms to APEX Migration

- Convert legacy **Oracle Forms to APEX** while maintaining business logic.
- Replace **PL/SQL triggers** with **APEX page processes**.
- Optimize **UI and performance** during migration.

9. Leveraging APEX APIs

- Use **APEX_JSON** and **APEX_WEB_SERVICE** for data exchange.
- Implement **APEX_MAIL** for sending emails dynamically.
- Utilize **APEX_COLLECTIONS** for managing session-based temporary data.

10. Advanced UI/UX Customization

- Customize themes using **Universal Theme & Theme Roller**.
- Implement **CSS and JavaScript frameworks** like Bootstrap & Tailwind.
- Use **Interactive Grid customization** for advanced data manipulation.

11. DevOps & Deployment Best Practices

- Automate APEX deployments using **Liquibase or SQLcl**.

- Implement **version control** for APEX applications.
- Use **APEX Export and Import** for CI/CD pipelines.